Practico 3

Números aleatorios y Método de Monte Carlo

Índice:

1. Ejercicio 1 - pág. 2
2. Ejercicio 2 - pág. 6
3. Ejercicio 3 - pág. 9
4. Ejercicio 4 - pág. 12
5. Ejercicio 5 - pág. 16
6. Ejercicio 6 - pág. 22
7. Ejercicio 7 - pág. 24
8. Ejercicio 8 - pág. 27
9. Ejercicio 9 - pág. 28

Ejercicio 1 (Índice)

Enunciado: Para el estudio mediante simulación es necesario generar muchos números aleatorios en la computadora. Estos corresponden a variables aleatorias uniformemente distribuidas en el intervalo $(0, 1)$.
Existen en la literatura varias rutinas portables, optimizadas para generar enormes cantidades de números pseudo-aleatorios con velocidad razonable.

- a) Determinar el período de la secuencia de von Neumann generada a partir de la semilla:
  1. 3009
  2. 7600
  3. 1234
  4. 4321
- b) Dar el valor de c y a para que cada generador tenga periodo máximo.
  1. $y_{i+1} = 5y_i + c \quad mod(2^5)$
  2. $x_{i+1} = ax_i \quad mod(31)$
  3. Considerar el generador $z_i = y_i + x_i \quad mod(2^5)$ y calcular su periodo.
  4. Representar en tres gráficos separados pares (y_i, y_{i+1}), (x_i, x_{i+1}) y (z_i, z_{i+1})
- c) Indicar en cuales de los siguientes casos el generador

  $$y_{i+1} = ay_i + c \quad mod(M)$$

  genera una secuencia de periodo máximo.
  1. $a = 125, c = 3, M = 2^9$
  2. $a = 123, c = 3, M = 2^9$
  3. $a = 5, c = 3, M = 71$
  4. $a = 7, c = 0, M = 71$
- d) Utilice el generador RANDU

  $$u_i = au_{i-1} \quad mod(M) \qquad a = 2^{16} + 3, M = 2^{31}$$

  para generar puntos aleatorios en el cubo $[0, M) \times [0, M) \times [0, M)$, de la forma:

  $$(u_1, u_2, u_3), (u_4, u_5, u_6), \ldots$$

  y estimar el porcentaje de puntos que caen en la esfera centrada en $(M/2, M/2, M/2)$ de radio $M/10$.
  Repetir el procedimiento con el generador

  $$y_i = ay_{i-1} \quad mod(M) \qquad a = 7^5, M = 2^{31} - 1$$

  ¿Cuál es los dos generadores estima mejor el valor real?

Solución:

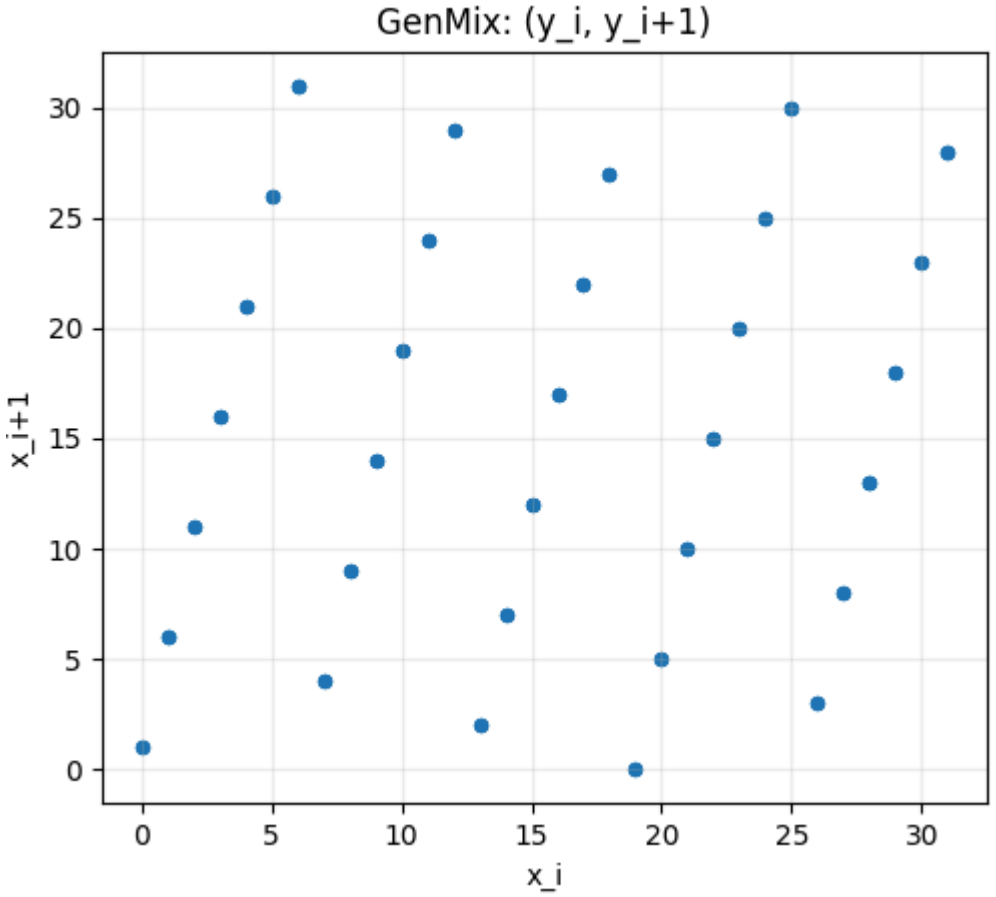
- a)
  1. Para 3009, el periodo es 5
  2. Para 7600, el periodo es 1
  3. Para 1234, el periodo es 57
  4. Para 4321, el periodo es 71
- b)
  1. Dado que es un generador mixto, debemos comprobar las propiedades para que tenga periodo 2^5.
     - $mcd(c, 2^5) = 1 \rightarrow c = 1$.
  2. Dado que es un generado multiplicativo, debemos comprobar las propiedades para que tenga periodo máximo.
     - a debe ser raíz primitiva de 31.
     - Tenemos que $30 = 2 \cdot 3 \cdot 5$.
     - Tomando $a = 3$, tenemos que

$$\begin{aligned} a^{30/2} &= a^{15} \not\equiv 1 \quad mod(31) \\ a^{30/3} &= a^{10} \not\equiv 1 \quad mod(31) \\ a^{30/5} &= a^{6} \not\equiv 1 \quad mod(31) \end{aligned}$$

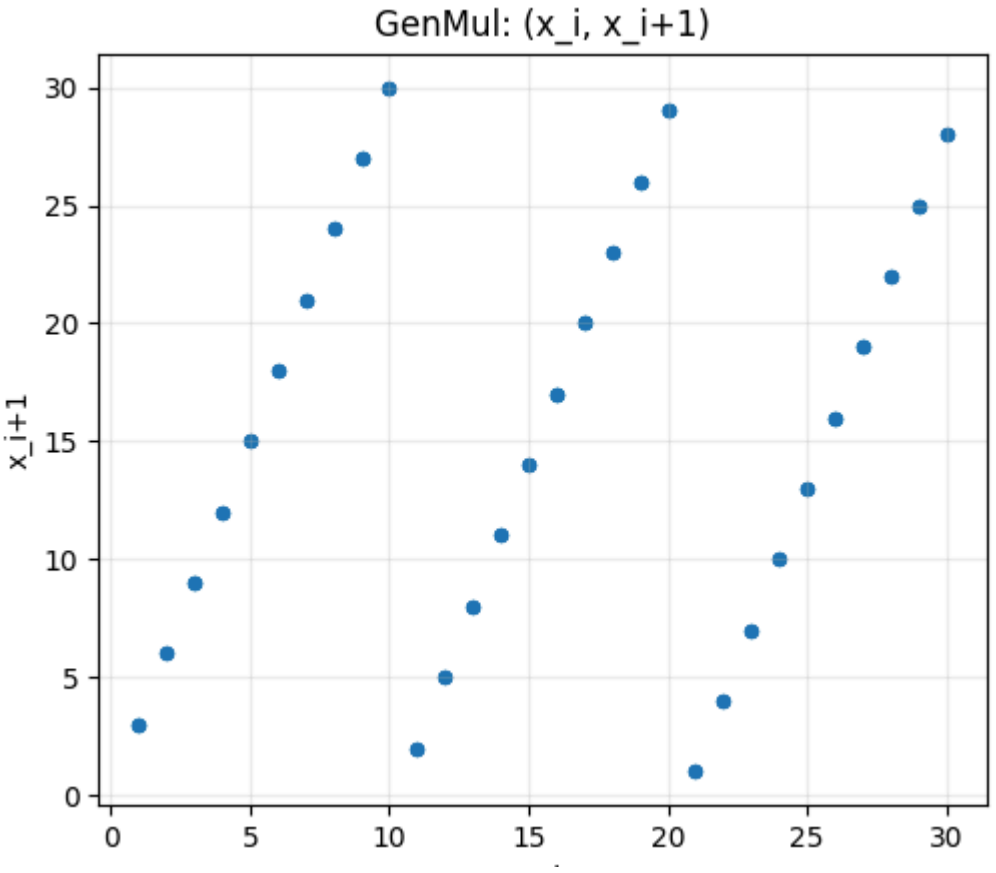
  3. Tenemos que y_i tiene periodo 32, x_i periodo 30, de modo que el periodo de la suma $y_i + x_i \quad mod(32)$ es

$$mcm(32, 30) = 480$$

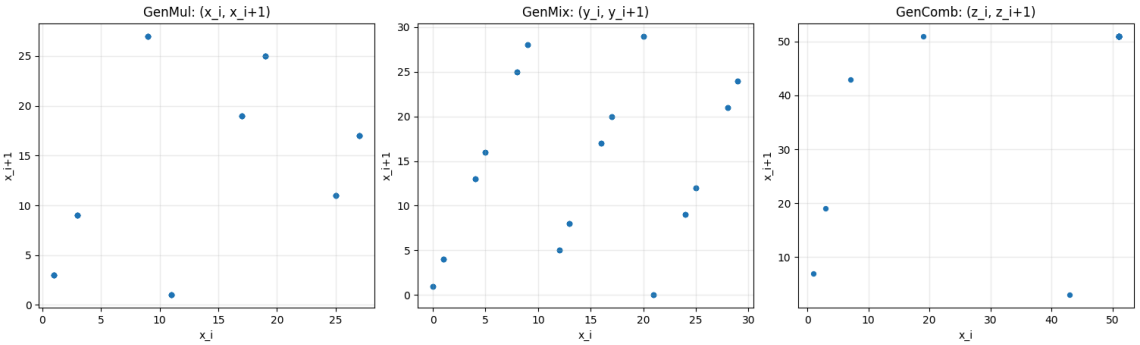
  4. Gráficos:
     - Para el generador y_i con $y_0 = 1, a = 5, c = 1, M = 32$ en $n = 100$ iteraciones,



- Para el generador x_i con $x_0 = 1, a = 3, c = 0, M = 31$ en $n = 100$ iteraciones,



- Para el generador z_i con $x_0 = y_0 = 1, a = 3, c = 1, M = 32$ en $n = 100$ iteraciones,



- c) Dado el generador $y_{i+1} = ay_i + c \quad mod(M)$,
  1. tipo mixto, tiene periodo $K = 2^9$
  2. tipo mixto, tiene periodo $K = 2^8 + 1$
  3. tipo multiplicativo, tiene periodo $K = 6$
  4. tipo multiplicativo, tiene periodo $K = 71$
- d) Teniendo en cuenta los siguientes datos

$$V_{esfera} = \frac{4}{3}\pi R^3 = \frac{4}{3}\pi\left(\frac{M}{10}\right)^3$$

Tenemos que el volumen del cubo $[0, M]^3$ es M^3, el porcentaje teórico(si los puntos fueran uniformes e independientes):

$$P_{teo} = \frac{V_{esfera}}{M^3} \cdot 100 = \frac{4\pi}{3} \cdot \frac{1}{1000} \cdot 100$$
$$P_{teo} = \frac{2\pi}{15} \approx 0.418879\%$$

Ahora, realizando simulaciones en ambos generadores para $u_0/y_0 = 1, 1234, 4321$ en $n = 100.000$ iteraciones, tenemos resultados

$$u_0 = y_0 = 1 \rightarrow \begin{cases} u_i \approx 0.378000\% \\ y_i \approx 0.413000\% \end{cases}$$
$$u_0 = y_0 = 1234 \rightarrow \begin{cases} u_i \approx 0.352000\% \\ y_i \approx 0.381000\% \end{cases}$$
$$u_i = y_i = 4321 \rightarrow \begin{cases} u_i \approx 0.384000\% \\ y_i \approx 0.422000\% \end{cases}$$

de modo que podemos concluir que el segundo generador es mejor.

Ejercicio 2 (Índice)

Enunciado: Se propone el siguiente juego en el cual todas las variables aleatorias que se generan son independientes e idénticamente distribuidas $\mathcal{U}(0,1)$: Se simula la variable aleatoria U. Si $U < \frac{1}{2}$, se suman dos nuevos números aleatorios $W_1 + W_2$. Pero si $U \geq \frac{1}{2}$, se suman tres números aleatorios. El resultado de la suma, en cualquiera de los casos, es una variable aleatoria X. Se gana el juego si $X \geq 1$.

- a) ¿Cuál es la probabilidad de ganar?
- b) Implementar un algoritmo en computadora que estime la probabilidad de ganar, esto es, la fracción de veces que se gana en n realizaciones del juego. Completar la siguiente tabla:

n	**100**	**1.000**	**10.000**	**100.000**	**1.000.000**
$P[X \geq 1]$					

Solución:

- a) Mediante probabilidad total, se pregunta

$$P(X \geq 1) = P(U < \frac{1}{2}) \cdot P(W_1 + W_2 \geq 1) + P(U \geq \frac{1}{2}) \cdot P(W_1 + W_2 + W_3 \geq 1)$$

  - Calculamos por partes,
    - $P(W_1 + W_2 \geq 1)$, $W_1 + W_2$ tiene distribución triangular con densidad

$$f_X(x) = \begin{cases} x & 0 \leq x \leq 1 \\ 2 - x & 1 \leq x \leq 2 \end{cases}$$

      por lo tanto

$$P(W_1 + W_2 \geq 1) = \int_1^2 2 - x \, dx = 0.5$$

    - $P(W_1 + W_2 + W_3 \geq 1)$, $W_1 + W_2 + W_3$ tiene distribución para el caso $n = 3$ de Irwin-Hall, su densidad es

$$f_X(x) = \begin{cases} \frac{1}{2}x^2 & 0 \leq x \leq 1 \\ \frac{1}{2}(-2x^2 + 6x - 3) & 1 \leq x \leq 2 \\ \frac{1}{2}(3 - x)^2 & 2 \leq x \leq 3 \end{cases}$$

por lo tanto

$$
\begin{aligned}
P(W_1 + W_2 + W_3 \geq 1) &= \int_1^2 \frac{1}{2}(-2x^2 + 6x - 3)\,dx + \int_2^3 \frac{1}{2}(3-x)^2\,dx \\
&= \frac{2}{3} + \frac{1}{6} \\
&= \frac{5}{6}
\end{aligned}
$$

- $P(U < \frac{1}{2}) = P(U \geq \frac{1}{2}) = \frac{1}{2}$

- De esta manera,

$$
P(X \geq 1) = \frac{1}{2} \cdot 0.5 + \frac{1}{2} \cdot \frac{5}{6} = \frac{2}{3}
$$

- b) Sea el algoritmo que utiliza la librería `random`, el cual utiliza en método de mersenne-twister para generar números aleatorios,

```python
import random,sys

def ejercicio2b(iterations: int):
    wins = 0
    for _ in range(iterations):
        u = random.random()
        if u < 0.5:
            w1 = random.random()
            w2 = random.random()
            s = w1 + w2
        else:
            w1 = random.random()
            w2 = random.random()
            w3 = random.random()
            s = w1 + w2 + w3
        if s ≥ 1:
            wins += 1
    print(f"Probability of winning: {wins / iterations:.4f}")


if __name__ == "__main__":
    n = int(sys.argv[1])
    if len(sys.argv) ≠ 2:
        print("Usage: python script.py <iterations>")
        sys.exit(1)
    elif n ≤ 0:
        print("Iterations must be a positive integer.")
        sys.exit(1)
    ejercicio2b(n)
```

tenemos que,

n	**100**	**1.000**	**10.000**	**100.000**	**1.000.000**
$P[X \geq 1]$	0.700	0.6660	0.6694	0.6657	0.6659

Ejercicio 3 (Índice)

Enunciado: Las maquinas tragamonedas usualmente generan un premio cuando hay un acierto. Supongamos que se genera el acierto con le siguiente esquema: se genera un número aleatorio, y

1. si es menor a un tercio, se suman dos nuevos número aleatorios
2. si es mayor o igual a un tercio, se suman tres números aleatorios.
   Si el resultado de la suma es menor o igual a 2, se genera un acierto.

- a) ¿Cuál es la probabilidad de acertar?
- b) Implementar un algoritmo en computadora que estime la probabilidad de acertar, esto es, la fracción de veces que se acierta en n realizaciones del juego. Completar la siguiente tabla:

n	100	1.000	10.000	100.000	1.000.000
$P[X \leq 2]$					

Solución:

Sean los números aleatorios U, W_1, W_2, W_3 generados en una $\mathcal{U}(0,1)$.
Sea la variable aleatoria X que representa la suma de los números aleatorios en cada caso.

- a) Se pregunta

$$P(X \leq 2) = P(U < 1/3) \cdot P(W_1 + W_2 \leq 2) + P(U \geq 1/3) \cdot P(W_1 + W_2 + W_3 \leq 2)$$

si tomamos $P(W_1 + W_2 \leq 2)$ y $P(W_1 + W_2 + W_3 \leq 2)$ de la siguiente manera,

$$P(X \leq 2) = P(U < 1/3) \cdot [1 - P(W_1 + W_2 > 2)] + P(U \geq 1/3) \cdot [1 - P(W_1 + W_2 + W_3 > 2)]$$

calculamos por partes utilizando las herramientas del ejercicio anterior,

- $P(W_1 + W_2 > 2)$:

$$P(W_1 + W_2 > 2) = 0$$

- $P(W_1 + W_2 + W_3 > 2)$:

$$P(W_1 + W_2 + W_3 > 2) = \int_2^3 \frac{1}{2}(3-x)^2 \, dx = \frac{1}{6}$$

- $P(U < 1/3)$:

$$P(U < 1/3) = \int_0^{1/3} 1 \, dx = \frac{1}{3}$$

- $P(U \geq 1/3)$:

$$P(U \geq 1/3) = 1 - P(U < 1/3) = 1 - \frac{1}{3} = \frac{2}{3}$$

luego

$$\begin{aligned} P(X \leq 2) &= \frac{1}{3} \cdot [1 - 0] + \frac{2}{3} \cdot [1 - \frac{1}{6}] \\ &= \frac{8}{9} \end{aligned}$$

- b) Sea el algoritmo que utiliza la librería `random`, el cual utiliza el método de mersenne-twister para generar números aleatorios,

```python
import random,sys

def ejercicio3b(iterations: int):
    hits = 0
    for _ in range(iterations):
        u = random.random()
        if u < (1/3):
            w1 = random.random()
            w2 = random.random()
            s = w1 + w2
        else:
            w1 = random.random()
            w2 = random.random()
            w3 = random.random()
            s = w1 + w2 + w3
        if s ≤2:
            hits += 1
    print(f"Probability of winning: {hits / iterations:.4f}")


if __name__ == "__main__":
    n = int(sys.argv[1])
    if len(sys.argv) ≠ 2:
        print("Usage: python script.py <iterations>")
        sys.exit(1)
    elif n ≤ 0:
        print("Iterations must be a positive integer.")
        sys.exit(1)
    ejercicio3b(n)
```

tenemos que,

n	**100**	**1.000**	**10.000**	**100.000**	**1.000.000**
$P[X \leq 2]$	0.9400	0.8890	0.8901	0.8886	0.8890

Ejercicio 4 (Índice)

Enunciado: Un supermercado posee 3 cajas. Por una cuestión de ubicación, el 40% de los clientes eligen la caja 1 para pagar, el 32% la caja 2, y el 28% la caja 3. El tiempo que espera una persona para ser atendida en cada caja distribuye exponencial con medias de $3, 4$ y 5 minutos respectivamente.

- a) ¿Cuál es la probabilidad de que un cliente espere menos de 4 minutos para ser atendido?
- b) Si el cliente tuvo que esperar más de 4 minutos. ¿Cuál es la probabilidad de que el cliente haya elegido cada una de las cajas?
- c) Simule el problema y estime las probabilidades anteriores con 1.000 iteraciones.

Solución:

Sean C_i el evento de que se elige la caja $i = 1, 2, 3$, cada una con $P(C_1) = 0.4$, $P(C_2) = 0.32$ y $P(C_3) = 0.28$ respectivamente.
Sea T_i el tiempo que espera $i = 1, 2, 3$ en que una persona es atendida, cada una con distribución $T_1 \sim \mathcal{E}(1/3)$, $T_2 \sim \mathcal{E}(1/4)$ y $T_3 \sim \mathcal{E}(1/5)$. Sea T el tiempo total de espera de un cliente.

- a) Se pregunta,

$$\begin{aligned} P(T < 4) &= P(C_1) \cdot P(T_1 < 4) + P(C_2) \cdot P(T_2 < 4) + P(C_3) \cdot P(T_3 < 4) \\ &= 0.4 \cdot (1 - e^{-4/3}) + 0.32 \cdot (1 - e^{-4/4}) + 0.28 \cdot (1 - e^{-4/5}) \\ &\approx 0.6510 \end{aligned}$$

- b) Se pregunta,

$$\begin{aligned} P(C_k \,|\, T > 4) &= \frac{P(C_k \,|\, T > 4)}{P(T > 4)} \qquad \text{por bayes} \\ &= \frac{P(T \geq 4 \,|\, C_k) \cdot P(C_k)}{P(T > 4)} \\ &= \frac{P(T_k > 4) \cdot P(C_k)}{P(T > 4)} \end{aligned}$$

  Ahora necesito calcular $P(T > 4)$:

$$P(T > 4) = 1 - (T \leq 4) \approx 1 - 0.6510 \approx 0.3489$$

  y ahora, calculo para cada caja C_i:

$$P(C_1 \,|\, T > 4) = \frac{P(T_1 > 4) \cdot P(C_1)}{P(T > 4)} \approx \frac{(e^{-4/3}) \cdot 0.4}{0.3489} \approx 0.3022$$
$$P(C_2 \,|\, T > 4) = \frac{P(T_2 > 4) \cdot P(C_2)}{P(T > 4)} \approx \frac{(e^{-4/4}) \cdot 0.32}{0.3489} \approx 0.3374$$
$$P(C_3 \,|\, T > 4) = \frac{P(T_3 > 4) \cdot P(C_3)}{P(T > 4)} \approx \frac{(e^{-4/5}) \cdot 0.28}{0.3489} \approx 0.3604$$

- c) Sea el algoritmo que utiliza la librería `numpy` para la generación de números aleatorios y calculo de números aleatorios que sigan una distribución exponencial,

```python
def ejercicio4c(n: int):
    menor4 = 0
    mayor4_por_caja = [0, 0, 0]

    for _ in range(n):
        u = np.random.random()
        if u < (0.4):
            caja = 0 # Caja 1
            media = 3
        elif u < (0.4 + 0.35):
            caja = 1 # Caja 2
            media = 4
        else:
            caja = 2 # Caja 3
            media = 5
        tiempo_espera = np.random.exponential(scale=media)

        if tiempo_espera < 4:
            menor4 += 1
        else:
            mayor4_por_caja[caja] += 1

    total_mayor4 = sum(mayor4_por_caja)
    prob_menor4 = menor4 / n

    if total_mayor4 > 0:
        prob_mayor4_c1 = mayor4_por_caja[0] / total_mayor4
        prob_mayor4_c2 = mayor4_por_caja[1] / total_mayor4
        prob_mayor4_c3 = mayor4_por_caja[2] / total_mayor4
    else:
        prob_mayor4_c1 = prob_mayor4_c2 = prob_mayor4_c3 = 0

    print(f"P(T < 4) ≈ {prob_menor4:.4f}")
    print(f"P(C1 | T>4) ≈ {prob_mayor4_c1:.4f}")
    print(f"P(C2 | T>4) ≈ {prob_mayor4_c2:.4f}")
    print(f"P(C3 | T>4) ≈ {prob_mayor4_c3:.4f}")


if __name__ == "__main__":
    if len(sys.argv) ≠ 2:
        print("Usage: python3 ej4.py <iterations>")
        sys.exit(1)
    elif int(sys.argv[1]) ≤ 0:
        print("Iterations must be a positive integer.")
        sys.exit(1)

    iteraciones = int(sys.argv[1])
    ejercicio4c(iteraciones)
```

tenemos que, para $n = 1.000$,

$$
\begin{aligned}
&P(T < 4) \approx 0.6410 \\
&P(C_1 \,|\, T > 4) \approx 0.2981 \\
&P(C_2 \,|\, T > 4) \approx 0.3482 \\
&P(C_3 \,|\, T > 4) \approx 0.3538
\end{aligned}
$$

Ejercicio 5 (Índice)

Enunciado: Calcule exactamente el valor de las siguientes integrales. Mediante una simulación de Monte Carlo con $n = 1000, 5000, 10.000$ iteraciones, calcule a su vez un valor aproximado y compare con el valor exacto.

- a) $\int_0^1 (1-x^2)^{3/2} dx$
- b) $\int_2^3 \frac{x}{x^2-1} dx$
- c) $\int_0^\infty x(1+x^2)^{-2} dx$
- d) $\int_{-\infty}^\infty e^{-x^2} dx$
- e) $\int_0^1 \left[\int_0^1 e^{(x+y)^2} dx\right] dy$
- f) $\int_0^\infty \left[\int_0^x e^{-(x+y)} dy\right] dx$

  Ayuda: Sea $I_y(x) = \begin{cases} 1 & \text{si } y < x \\ 0 & \text{si } y \geq x \end{cases}$. Utilice esta función para igualar la integral del ítem f) a otra cuyos términos vayan de 0 a ∞.

  Completar la siguiente tabla:

n\ integral $\rightarrow$	(a)	(b)	(c)	(d)	(e)	(f)
100						
1.000						
10.000						
100.000						
1.000.000						

Solución:

- a) El valor exacto es, $\int_0^1 (1-x^2)^{3/2}\, dx = \frac{3\pi}{16}$, mediante Montecarlo, sean las variables aleatorias uniformes independientes, $U_1 = u_1, U_2 = u_2, \ldots, U_N = u_N$, aproximamos,

$$\int_0^1 (1-x^2)^{3/2}\, dx \sim \frac{1}{N} \sum_{i=1}^{N} (1-u_i^2)^{3/2}$$

- b) El valor exacto es, $\int_2^3 \frac{x}{x^2-1} dx = \frac{1}{2} \cdot (ln(8) - ln(3))$, mediante Montecarlo, tenemos el caso de el intervalo (a, b), en este caso $a = 2$ y $b = 3$, realizamos la transformación de

la función para llevarla al intervalo $(0,1)$,

$$\int_2^3 \frac{x}{x^2-1}dx = \int_0^1 \frac{y+2}{(y+3)(y+1)}\,dy$$

,ahora sean las variables aleatorias uniformes independientes,
$U_1 = u_1, U_2 = u_2, \ldots, U_N = u_N$, aproximamos,

$$\int_0^1 \frac{y+2}{(y+3)(y+1)}\,dy \sim \frac{1}{N}\sum_{i=1}^{N} \frac{u_i+2}{(u_i+3)(u_i+1)}$$

- c) El valor exacto es, $\displaystyle\int_0^\infty x(1+x^2)^{-2}dx = \frac{1}{2}$, mediante Montecarlo, tenemos el caso de el intervalo $(0,\infty)$, realizamos la transformación de la función para llevarla al intervalo $(0,1)$,

$$\int_0^\infty x(1+x^2)^{-2}dx = \int_0^1 \frac{(1-y)y}{(2y^2-2y+1)^2}\,dy$$

, ahora sean las variables aleatorias uniformes independientes,
$U_1 = u_1, U_2 = u_2, \ldots, U_N = u_N$, aproximamos,

$$\int_0^1 \frac{(1-y)y}{(2y^2-2y+1)^2}\,dy \sim \frac{1}{N}\sum_{i=1}^{N} \frac{(1-u_i)u_i}{(2u_i^2-2u_i+1)^2}$$

- d) El valor exacto es $\displaystyle\int_{-\infty}^\infty e^{-x^2}dx = \sqrt{\pi}$, mediante Montecarlo, podemos reescribir la integral de la siguiente manera,

$$\int_{-\infty}^\infty e^{-x^2}dx = 2\int_0^\infty e^{-x^2}dx$$

y así tenemos el caso de $(0,\infty)$ de Montecarlo, realizamos la transformación de la función,

$$2\int_0^\infty e^{-x^2}dx = 2\int_0^1 exp\left(-\frac{(1-y)^2}{y^2}\right)\cdot\left(\frac{1}{y^2}\right)dy$$

, ahora sean las variables aleatorias uniformes independientes,
$U_1 = u_1, U_2 = u_2, \ldots, U_N = u_N$, aproximamos,

$$2\int_0^1 exp\left(-\frac{(1-y)^2}{y^2}\right)\cdot\left(-\frac{1}{y^2}\right)dy \sim \frac{2}{N}\sum_{i=1}^{N} exp\left(-\frac{(1-u_i)^2}{u_i^2}\right)\cdot\left(-\frac{1}{u_i^2}\right)$$

- e) El valor exacto es
$\displaystyle\int_0^1\left[\int_0^1 e^{(x+y)^2}dx\right]dy = e - \frac{1}{2} - \frac{e^4}{2} + \sqrt{\pi}(erfi(2) - erfi(1)) \approx 4.8922$ donde
$\displaystyle erfi(z) = \frac{2}{\sqrt{\pi}}\int_0^z e^{t^2}dt$, mediante Montecarlo, tenemos el caso de integrales múltiples, pero precisamente tenemos el intervalo de las integrales a nuestro favor, por lo tanto,

solo debemos aplicar la transformación de la función,

$$\int_0^1 \left[\int_0^1 e^{(x+y)^2} dx\right] dy \sim \frac{1}{N}\sum_{i=1}^{N} e^{(u_{i^1}+u_{i^2})^2}$$

para dos conjuntos de variables aleatorias uniformes independientes.

- f) El valor exacto es $\int_0^\infty \left[\int_0^x e^{-(x+y)} dy\right] dx = \frac{1}{2}$, mediante Montecarlo, necesitamos usar la función indicadora para realizar una transformación en un intervalo,

$$\int_0^\infty \left[\int_0^x e^{-(x+y)} dy\right] dx = \int_0^\infty \left[\int_0^\infty e^{-(x+y)} \cdot \mathbb{I}_y(x)\, dy\right] dx$$

, ahora podemos interpretar $e^{-x}e^{-y}$ como densidad de dos exponenciales independientes de tasa 1(en $x > 0, y > 0$):

$$\int_0^\infty \left[\int_0^\infty e^{-(x+y)} \cdot \mathbb{I}_y(x)\, dy\right] dx = E[\mathbb{I}_y(x)]$$

, sean las variables aleatorias independientes u_{i^1} y u_{i^2} con distribución $\mathcal{E}(1)$, entonces,

$$\int_0^\infty \left[\int_0^\infty e^{-(x+y)} \cdot \mathbb{I}_y(x)\, dy\right] dx \sim \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}_y(x)$$

tal que,

$$\mathbb{I}_y(x) = \begin{cases} 1 & u_{i^1} < u_{i^2} \\ 0 & \text{caso contrario} \end{cases}$$

Ahora con los cálculos, completamos la tabla,

n\ integral $\rightarrow$	**(a)**	**(b)**	**(c)**	**(d)**	**(e)**	**(f)**
100	0.5162	0.4931	0.4384	1.9541	4.5747	0.6000
1.000	0.5701	0.4824	0.4957	1.7669	4.8408	0.5070
10.000	0.5910	0.4914	0.5032	1.7553	4.9566	0.4957
100.000	0.5888	0.4902	0.5015	1.7726	4.9091	0.5039
1.000.000	0.5891	0.4903	0.5000	1.7732	4.8933	0.4997

este fue el algoritmo que se utilizo,

```python
import sys
import numpy as np

def ejercicio5a(n: int):
    sum = 0
    for _ in range(n):
        u = np.random.uniform(0, 1)
        sum += (1-u**2)**(3/2)
    rta = sum/n
    return rta

def ejercicio5b(n: int):
    sum = 0
    for _ in range(n):
        u = np.random.uniform(0, 1)
        sum += (u+2)/((u+3)*(u+1))
    rta = sum/n
    return rta

def ejercicio5c(n: int):
    sum = 0
    for _ in range(n):
        u = np.random.uniform(0, 1)
        sum += ((1-u)*u)/((2*(u**2)-2*u+1)**2)
    rta = sum/n
    return rta

def ejercicio5d(n: int):
    sum = 0
    for _ in range(n):
        u = np.random.uniform(0, 1)
        sum += np.exp(-(1-u)**2/(u**2)) * ((1)/(u**2))

    rta = 2*sum/n
    return rta
```

```python
def ejercicio5e(n: int):
    sum = 0
    for _ in range(n):
        u1 = np.random.uniform(0, 1)
        u2 = np.random.uniform(0, 1)
        sum += np.exp((u1+u2)**2)

    rta = sum/n
    return rta

def ejercicio5f(n: int):
    sum = 0
    for _ in range(n):
        u1 = np.random.exponential(1)
        u2 = np.random.exponential(1)
        if (u1 < u2):
            sum += 1
        else:
            sum += 0

    rta = sum/n
    return rta
```

```python
if __name__ == "__main__":
    if len(sys.argv) ≠ 2:
        print("Usage: python ej5.py <n>")
        sys.exit(1)
    elif int(sys.argv[1]) < 0:
        print("n must be a non-negative integer")
        sys.exit(1)
    n = int(sys.argv[1])

    a = ejercicio5a(n)
    b = ejercicio5b(n)
    c = ejercicio5c(n)
    d = ejercicio5d(n)
    e = ejercicio5e(n)
    f = ejercicio5f(n)

    print(f"En {n} iteraciones:")
    print(f"Ejercicio 5a: {a:.4f}")
    print(f"Ejercicio 5b: {b:.4f}")
    print(f"Ejercicio 5c: {c:.4f}")
    print(f"Ejercicio 5d: {d:.4f}")
    print(f"Ejercicio 5e: {e:.4f}")
    print(f"Ejercicio 5f: {f:.4f}")
```

Ejercicio 6 (Índice)

Enunciado: Es posible aproximar el valor de π calculando el área de un círculo de radio 1 centrado en 0. Para eso, se necesitan generar N puntos aleatorios en la caja $[-1,1] \times [-1,1]$ y contar la cantidad de veces que los mismos caen dentro del círculo. El cociente entre número y N, multiplicando por 4(el área del cuadrado donde está contenido el cirulo) es una aproximación de π.

Completar la siguiente tabla con los valores obtenidos para distintos N y compararlos con `numpy.pi` o `math.pi`:

n	π
1.000	
10.000	
100.000	

Solución:

Mediante el siguiente algoritmo,

```python
import numpy as np
import sys


def valorpi(n: int):
    hit = 0
    for _ in range(n):
        u = 2 * np.random.rand() - 1
        v = 2 * np.random.rand() - 1
        if u**2 + v**2 ≤ 1:
            hit += 1
    return 4 * hit / n


if __name__ == "__main__":
    if len(sys.argv) ≠ 2:
        print("Usage: python ej5.py <n>")
        sys.exit(1)
    elif int(sys.argv[1]) < 0:
        print("n must be a non-negative integer")
        sys.exit(1)
    n = int(sys.argv[1])

    pi_estimate = valorpi(n)
    print(f"En {n} iteraciones:")
    print(f"Valor estimado de pi: {pi_estimate}")
    print(f"Valor real de pi en numpy: {np.pi}")
    print(f"Error absoluto: {abs(pi_estimate - np.pi)}")
```

llenamos la tabla, pero con un campo extra que calcula el valor absoluto de la diferencia entre el valor real de `numpy.pi` y el estimado,

n	π	Diferencia
1.000	3.088	0.05359265
10.000	3.1548	0.01320734
100.000	3.14484	0.00324734

Ejercicio 7 (Índice)

Enunciado: Para $U_1, U_2, \ldots$ variables aleatorias uniformemente distribuidas en el intervalo $(0, 1)$, se define:

$$N = Minimo\{n : \sum_{i=1}^{n} U_i > 1\}$$

Es decir, N es igual a la cantidad de números aleatorios que deben sumarse para exceder a 1.

- a) Estimar $E[N]$ generando n valores de N y completar la siguiente tabla:

n	**100**	**1.000**	**10.000**	**100.000**	**1.000.000**
$E[N]$					

- b) Calcular el valor exacto de $E[N]$.

Solución:

- a) Mediante el siguiente algoritmo, estimamos $E[N]$,

```python
import sys
import numpy as np

def simular_N():
    sum = 0
    count = 0

    while sum ≤ 1:
        sum += np.random.uniform(0, 1)
        count += 1
    return count


def estimar(n: int):
    return np.mean([simular_N() for _ in range(n)])

if __name__ == "__main__":
    if len(sys.argv) ≠ 2:
        print("Uso: python ej7.py <N>")
        sys.exit(1)
    elif int(sys.argv[1]) ≤ 0:
        print("N debe ser un entero positivo.")
        sys.exit(1)
    N = int(sys.argv[1])

    valor_estimado = estimar(N)

    print(f"En {N} iteraciones:")
    print(f"Valor estimado de E[N]: {valor_estimado}")
```

y completamos la tabla,

n	100	1.000	10.000	100.000	1.000.000
$E[N]$	2.8	2.718	2.7152	2.71775	2.718282

- b) Sabiendo que $P(N > k) = P(U_1 + U_2 + \ldots + U_k \leq 1)$, para k uniformes idénticamente distribuidas en $(0, 1)$, el volumen de la región $\sum_{i=1}^{k} u_i \leq 1$ en el cubo $[0, 1]^k$ es $\frac{1}{k!}$, entonces

$$P(N > k) = \frac{1}{k!}, \qquad k = 0, 1, 2, \ldots$$

luego,

$$E[N] = \sum_{k=0}^{N} P(N > k) = \sum_{k=0}^{N} \frac{1}{k!} = e$$

, por lo tanto el valor exacto es

$$E[N] = e$$

Ejercicio 8 (Índice)

Enunciado: Para $U_1, U_2, \ldots$ números aleatorios, se define:

$$N = Maximo\{n : \prod_{i=1}^{n} U_i \geq e^{-3}\}$$

donde: $\prod_{i=1}^{0} U_i = 1$. Mediante n simulaciones determinar:

- a)

n	100	1.000	10.000	100.000	1.000.000
E[N]					

- b) $P(N = i)$ para $i = 0, 1, 2, 3, 4, 5, 6$, usando $n = 1.000.000$.

Solución:

No tengo ni la mas remota idea...

Ejercicio 9 (Índice)

Enunciado: Un juego consiste en dos pasos. En el primer paso se tira un dado convencional. Si sale 1 o 6 tira un nuevo dado y se le otorga al jugador como puntaje el doble del resultado obtenido en esta nueva tirada; pero si sale $2, 3, 4$ o 5 en la primer tirada, el jugador debería tirar dos nuevos dados, y recibiría como puntaje la suma de los dados. Si el puntaje del jugador excede los 6 puntos, entonces gana.

- a) Realizar un cálculo teórico de la probabilidad de que un jugador gane.
- b) Estime la probabilidad de que un jugador gane mediante una simulación.

Solución:

Sea $D \sim \mathcal{U}\{1, 2, 3, 4, 5, 6\}$, tal que $P(D = k) = \frac{1}{6}$ y sea S el resultado de sumar en cada caso.
Sea D_k el resultado de la tirada k.

- a) Se pregunta,

  $P(S > 6) = P(S > 6 \,|\, D_1 \in \{1, 6\}) \cdot P(D_1 \in \{1, 6\}) + P(S > 6 \,|\, D_2 \in \{2, 3, 4, 5\}) \cdot P(D_2 \in \{2, 3,$

  Calculamos por casos,
  - $P(S > 6 \,|\, D_1 \in \{1, 6\})$:

    $$P(S > 6 \,|\, D_1 \in \{1, 6\}) = P(2D_2 > 6) = P(D_2 > 3) = P(D_2 \in \{4, 5, 6\}) = \frac{3}{6} = \frac{1}{2}$$

  - $P(S > 6 \,|\, D_1 \in \{2, 3, 4, 5\})$:

    $$P(S > 6 \,|\, D_1 \in \{2, 3, 4, 5\}) = P(D_2 + D_3 > 6) = \frac{21}{36} = \frac{7}{12}$$

  - $P(D_1 \in \{1, 6\})$:

    $$P(D_1 \in \{1, 6\}) = \frac{2}{6} = \frac{1}{3}$$

  - $P(D_2 \in \{2, 3, 4, 5\})$:

    $$P(D_2 \in \{2, 3, 4, 5\}) = \frac{4}{6} = \frac{2}{3}$$

    Por lo tanto,

    $$P(S > 6) = \frac{1}{2} \cdot \frac{1}{3} + \frac{7}{12} \cdot \frac{2}{3} = \frac{1}{6} + \frac{7}{18} = \frac{5}{9}$$

- b) Sea el algoritmo siguiente,

```python
import sys
import numpy as np

def ejercicio9b(n: int):
    win = 0

    for _ in range(n):
        d1 = np.random.randint(1, 7)
        if d1 in (1,6):
            d2 = np.random.randint(1, 7)
            points = 2*d2
        else:
            d3 = np.random.randint(1, 7)
            d4 = np.random.randint(1, 7)
            points = d3 + d4
        if points > 6:
            win+=1

    return win/n

if __name__ == "__main__":
    if len(sys.argv) ≠ 2:
        print("Uso: python ej9.py <N>")
        sys.exit(1)
    elif int(sys.argv[1]) ≤ 0:
        print("N debe ser un entero positivo.")
        sys.exit(1)
    N = int(sys.argv[1])

    estimate = ejercicio9b(N)
    print(f"En {N} iteraciones:")
    print(f"Probabilidad estimada de ganar: {estimate:.4f}")
```

Completamos la siguiente tabla para las respectivas iteraciones,

n	**100**	**1.000**	**10.000**	**100.000**	**1.000.000**
$P(S > 6)$	0.6000	0.5670	0.5497	0.5557	0.5557